
Kilo12

Diseño Técnico

Versión	1.3
Fecha	2026-09-22
Estado	Propuesta de diseño para revisión. No se ha escrito código de aplicación todavía.
Documento base	requerimientos-funcionales.md v1.4

Historial

Cambios en 1.3: se incorporan la libra como unidad base, los tres métodos de pago y la tasa de cambio congelada por venta. El esquema añade el arqueo por moneda y el modo de cierre.

Cambios en 1.2: se cierra DA-2 con la convención de nomenclatura del proyecto, detallada en DT-13: el dominio se escribe en español y la estructura técnica en inglés.

Cambios en 1.1: se cierra DA-1 (no se firma el ejecutable, por ser uso personal) y se detallan sus consecuencias en DT-12, junto con el requisito de incorporar WebView2 al instalador de Windows.

Contenido

1. Propósito de este documento	1
2. Restricciones que manda el negocio	2
3. Decisiones técnicas	3
3.1. DT-1 — La lógica de negocio vive en Rust; el frontend solo presenta	3
3.2. DT-2 — SQLite como motor de persistencia	3
3.3. DT-3 — rusqlite con SQLite compilado dentro del binario	4
3.4. DT-4 — Dinero y cantidades como enteros escalados	4
3.5. DT-5 — Arquitectura hexagonal en el núcleo	5
3.6. DT-6 — Angular con componentes independientes y señales	5
3.7. DT-7 — Contrato entre Rust y Angular	6
3.8. DT-8 — Esquema de base de datos	6
3.9. DT-9 — Transaccionalidad	9
3.10.DT-10 — Migraciones de esquema	10
3.11.DT-11 — Respaldo y restauración	10
3.12.DT-12 — Compilación y distribución	10
3.13.DT-13 — Nomenclatura: el dominio habla español	11
4. Estructura del repositorio	13
5. Modelo de dominio	14
6. Estrategia de pruebas	15
7. Riesgos técnicos	16
8. Decisiones	17
8.1. Cerradas	17
8.2. Abiertas	17
9. Trazabilidad: requerimiento → decisión	18

1 Propósito de este documento

Los requerimientos funcionales definen **qué** hace Kilo12. Este documento define **cómo** se construye, y sobre todo **por qué se eligió cada opción**.

Toda decisión aquí lleva su justificación y las alternativas que se descartaron. Cuando dentro de seis meses alguien —tú incluido— se pregunte por qué el dinero no se calcula en el frontend o por qué el inventario no usa un archivo JSON, la respuesta está escrita.

El diseño se somete a los requerimientos, no al revés. Cada decisión técnica se rastrea hasta el requerimiento que la obliga (§9).

2 Restricciones que manda el negocio

Estas no se negocian: vienen del documento de requerimientos y condicionan todo lo demás.

Origen	Restricción	Consecuencia técnica
R-1	Operación 100 % offline	Ninguna dependencia de red en tiempo de ejecución. Todo el código y los recursos viajan dentro del binario.
R-2	Almacenamiento local, sin servidor	Base de datos embebida en el equipo del usuario.
R-6	El equipo puede fallar y no hay nube	El respaldo debe ser trivial de ejecutar y de verificar.
RNF-5	Ninguna operación puede dejar el inventario inconsistente	Persistencia transaccional con garantías ACID reales.
RNF-6	Aritmética decimal exacta para dinero	Prohibido el punto flotante binario en todo el recorrido del dato.
RNF-1	Búsqueda en menos de 100 ms con 5 000 productos	Índices adecuados y consultas que no recorran tablas completas.
RNF-2	Arranque en menos de 3 s	Sin trabajo pesado en el inicio; migraciones rápidas e incrementales.
RNF-4	Poder añadir lector y impresora sin rediseñar	Los periféricos entran por puertos, no por el medio del dominio.
RNF-8	Windows principal, macOS secundaria	Compilación multiplataforma desde el primer día.

3 Decisiones técnicas

3.1 DT-1 — La lógica de negocio vive en Rust; el frontend solo presenta

Decisión. Todas las reglas —valoración, existencias, presentaciones, comisión, arqueo— se implementan y se ejecutan en Rust. Angular muestra datos y recoge entradas; no calcula dinero ni decide nada del dominio.

Por qué. RNF-6 exige aritmética decimal exacta. JavaScript no tiene enteros ni decimales exactos para este uso: todo número es f64 binario, donde $0.1 + 0.2 \neq 0.3$. Calcular allí el costo promedio ponderado o la comisión introduce un error que se acumula operación tras operación hasta descuadrar el inventario por cantidades que nadie sabe explicar. El dinero se calcula donde hay tipos exactos.

Consecuencia importante. Los montos y cantidades cruzan la frontera hacia Angular **como cadenas de texto, nunca como números JSON** (ver DT-7). Un `number` en JSON es f64: serializar 41.67 y deserializarlo puede no devolver 41.67.

Alternativas descartadas. Repartir cálculos entre ambos lados: duplica reglas, y la duplicación de reglas es donde nacen las discrepancias que nadie reproduce.

3.2 DT-2 — SQLite como motor de persistencia

Decisión. SQLite embebido, compilado dentro del binario.

Por qué.

- **Transaccional de verdad.** RNF-5 exige que una venta no pueda dejar el inventario a medias. SQLite da atomicidad real; un archivo escrito a mano, no.
- **Un solo archivo.** Esto convierte RF-DAT-02 (respaldo) en copiar un archivo y RF-DAT-03 (restauración) en reemplazarlo. La operación más crítica del sistema resulta ser la más simple.
- **Sin servidor ni instalación.** Coherente con R-2 y con RNF-10.
- **Consultas.** Los informes de §6.13 de los requerimientos son agregaciones por fecha, producto, categoría y método de pago. SQL las resuelve; un almacén clave-valor obligaría a recorrer todo en memoria.

Alternativas descartadas.

Opción	Motivo del descarte
Archivos JSON o CSV	Sin transacciones. Un corte de luz a mitad de escritura corrompe el archivo completo. Inaceptable con R-6.
sled / redb (clave-valor)	Rápidos y en Rust puro, pero sin consultas. Cada informe habría que resolverlo recorriendo y agregando a mano.
Motor cliente-servidor (PostgreSQL, MySQL)	Exige instalar y mantener un servicio. Contradice R-2 y RNF-10 frontalmente.

3.3 DT-3 — rusqlite con SQLite compilado dentro del binario

Decisión. Acceso mediante `rusqlite` con la característica `bundled`.

Por qué. `bundled` compila el código fuente de SQLite dentro del ejecutable: la aplicación no depende de ninguna biblioteca instalada en el sistema, lo que cumple RNF-10 y evita el problema clásico de versiones distintas de SQLite entre Windows y macOS.

Al ser una aplicación de un solo usuario (R-3), no hay contención: una única conexión protegida por exclusión mutua es suficiente y elimina toda la complejidad de un pool. Las operaciones se ejecutan en el hilo de bloqueo que ofrece Tauri para no congelar la interfaz.

Alternativas descartadas. `sqlx` aporta verificación de consultas en tiempo de compilación, pero exige una base de datos disponible durante la compilación, lo que complica el pipeline multiplataforma sin resolver ningún problema que tengamos. `diesel` impone su propio lenguaje de consulta y una curva que no se justifica en un esquema de este tamaño.

Configuración obligatoria de la conexión.

```
PRAGMA foreign_keys = ON;      -- SQLite las ignora por defecto: trampa clásica
PRAGMA journal_mode = WAL;    -- lecturas concurrentes y mejor resistencia a cortes
PRAGMA synchronous = FULL;    -- prioriza no perder datos sobre velocidad de escritura
PRAGMA busy_timeout = 5000;
```

`synchronous = FULL` es más lento que `NORMAL`, y es deliberado: en un equipo doméstico sin batería de respaldo, perder la última venta por un corte de luz es peor que tardar unos milisegundos más.

3.4 DT-4 — Dinero y cantidades como enteros escalados

Decisión. Ningún valor monetario ni de cantidad se almacena como decimal ni como flotante. Se almacenan como enteros de 64 bits con escala fija:

Magnitud	Escala	Unidad almacenada	Ejemplo
Dinero	10^{-6}	millonésimas de la moneda	\$41.67 → 41670000
Cantidad	10^{-3}	milésimas de la unidad base	1.250 lb → 1250
Factor de conversión	10^{-3}	milésimas	six-pack, factor 6 → 6000
Porcentaje de comisión	10^{-4}	diezmilésimas	2% → 20000
Tasa de cambio	10^{-6}	millonésimas de peso por dólar	700 CUP/USD → 700000000

La aritmética se implementa **directamente sobre los enteros**, calculando los productos intermedios en 128 bits y reescalando con redondeo al más cercano. No se usa ninguna biblioteca decimal: habiendo decidido enteros escalados, un tipo decimal intermedio solo añadiría una dependencia y conversiones en ambos sentidos. El resultado es un dominio sin dependencias externas, que es exactamente lo que DT-5 persigue.

Por qué esta escala. El costo promedio ponderado produce divisiones que no son exactas: $\$250 \div 6 = \$41.6666\dots$. Con dos decimales, cada operación pierde hasta medio centavo, y RF-COS-09 exige explícitamente que ese error no se acumule. Seis decimales dejan el error por operación por debajo de una millonésima; ni con cientos de miles de movimientos llega a afectar un centavo.

El rango de un entero de 64 bits con escala 10^{-6} cubre hasta $\pm 9,2$ billones de unidades monetarias. Un mercadito no se acerca ni de lejos.

Por qué enteros y no texto. Guardar el decimal como cadena también sería exacto, pero impediría que

SQL sumara: cada informe tendría que traer todas las filas a memoria y agregarlas a mano. Con enteros, `SUM()` es exacto y el trabajo lo hace el motor.

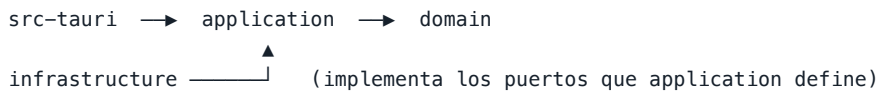
Dónde se redondea. Solo en dos puntos, y ambos quedan registrados: el importe que se le cobra al cliente (a los decimales de la moneda configurada) y los valores que se muestran en pantalla. El valor acumulado del inventario nunca se redondea.

3.5 DT-5 — Arquitectura hexagonal en el núcleo

Decisión. El código Rust se organiza como un espacio de trabajo de Cargo con tres capas, más el adaptador de Tauri:

crates/	
domain/	Entidades, objetos de valor, reglas e invariantes. Sin dependencias salvo <code>rust_decimal</code> . No sabe que existe SQLite.
application/	Casos de uso y PUERTOS (traits). Orquesta el dominio. No sabe qué implementa los puertos.
infrastructure/	ADAPTADORES: implementación SQLite de los puertos, migraciones, respaldo.
src-auri/	Adaptador de entrada: comandos Tauri, ensamblado de dependencias, ciclo de vida de la aplicación.

Regla de dependencia, innegociable: las flechas apuntan siempre hacia adentro.



`domain` no importa nada de las otras capas. Si algún día alguien escribe `use rusqlite` dentro de `domain`, el diseño se rompió.

Por qué. Tres razones concretas, no ideológicas:

1. **RNF-4** pide poder añadir lector de código de barras e impresora térmica sin rediseñar. Con puertos, cada periférico es un adaptador nuevo; el dominio ni se entera. 2. **Las reglas se prueban sin base de datos.** Los casos de prueba que los requerimientos ya dejaron escritos —el arroz de §6.6, el refresco de §6.2, la comisión de §6.11— son pruebas unitarias puras, en milisegundos, sin montar nada. 3. **Cambiar de motor de persistencia** significaría escribir otro adaptador, no tocar las reglas.

El riesgo honesto: para un proyecto de este tamaño, cuatro crates añaden ceremonia. Se asume a conciencia porque el núcleo —valoración, existencias, presentaciones— es lógica de negocio densa que conviene tener aislada y bajo prueba.

3.6 DT-6 — Angular con componentes independientes y señales

Decisión. Angular moderno: componentes standalone, señales para el estado, y separación entre componentes contenedores y de presentación.

Por qué. Es el terreno conocido del equipo, y el costo que suele pagarse por Angular —el peso del paquete— aquí no aplica igual: los recursos se sirven desde el propio binario, sin red de por medio. El arranque queda holgadamente dentro de los 3 s del RNF-2.

Organización por funcionalidad (estructura que grita lo que hace la aplicación, no qué framework usa):

ui/src/app/	
core/	Pasarela hacia Tauri, manejo de errores, configuración

shared/	Componentes de presentación reutilizables
features/	
venta/	La pantalla crítica: debe operarse sin ratón (RNF-3)
catalogo/	
inventario/	
vitrina/	
compras/	
caja/	
informes/	
ajustes/	

Estado. Señales dentro de cada funcionalidad. No se introduce una biblioteca de gestión de estado global: el estado real vive en SQLite, y la interfaz es una vista sobre él. Duplicarlo en memoria solo crea dos verdades que se contradicen.

La pantalla de venta manda. RNF-3 exige operar toda la venta con teclado. Eso condiciona su diseño: foco gestionado explícitamente, atajos, y ningún flujo que dependa de un clic.

3.7 DT-7 — Contrato entre Rust y Angular

Decisión. La comunicación se hace por comandos de Tauri. Cada caso de uso expone un comando con objetos de transferencia propios, separados de las entidades del dominio.

Reglas del contrato:

1. **Los montos y cantidades viajan como cadenas de texto.** Consecuencia directa de DT-1: un number de JSON es f64 y no preserva los valores exactos. 2. **Los objetos de transferencia no son entidades.** El dominio puede evolucionar sin romper la interfaz, y la interfaz no obliga a exponer lo que no necesita. 3. **Los errores son tipados,** nunca cadenas sueltas. Un enum de errores de aplicación se serializa con código y mensaje, para que la interfaz pueda distinguir «existencia insuficiente» de «error de base de datos» y reaccionar distinto. 4. **Los comandos son verbos del negocio,** no operaciones de base de datos: registrar_venta, reabastecer_vitrina, cerrar_caja. No existe un comando ejecutar_sql.

Forma del error:

```
{
  "codigo": "EXISTENCIA_INSUFICIENTE",
  "mensaje": "La vitrina tiene 3 unidades y se intentan vender 6",
  "detalle": { "producto_id": 42, "disponible": "3.000", "solicitado": "6.000" }
}
```

3.8 DT-8 — Esquema de base de datos

Principios aplicados:

- **El kárdex es un registro inmutable.** Los movimientos no se modifican ni se borran: una corrección es un movimiento nuevo en sentido contrario. Es lo que hace auditable el inventario (RF-INV-03).
- **La cantidad total no se duplica.** Se almacena la existencia por ubicación; la existencia total es su suma. El valor total del inventario sí se almacena, porque no es derivable de las cantidades.
- **Las fechas se guardan en UTC** con formato ISO-8601. La sesión de caja guarda además su fecha local de negocio, porque «el día» del mercadito lo define el dueño, no el meridiano.

Tablas principales

Tabla	Contenido	Requerimientos
categoria	Categorías administrables	RF-CAT-08
producto	Datos del producto, unidad base, valor total del inventario, stock mínimo	RF-CAT-01, RF-COS-02
presentacion	Nombre, factor de conversión, precio, activa, código de barras	RF-PRS-02, RF-CAT-12
existencia	Cantidad por (producto, ubicación)	RF-INV-01
movimiento_inventario	Kárdex inmutable de todo cambio de existencia	RF-INV-03, RF-INV-05
compra / compra_linea	Entradas de mercancía con su costo	RF-COM-01, RF-COM-02
venta / venta_linea	Ventas con precio y costo congelado por línea	RF-VTA-17, RF-VTA-13
devolucion / devolucion_linea	Devoluciones, independientes de la venta original	RF-VTA-16
sesion_caja	Apertura, cierre, arqueo, operador y comisión liquidada	RF-CAJ-01, RF-CMS-06
movimiento_efectivo	Entradas y salidas de efectivo ajenas a la venta	RF-CAJ-03
operador	Personas que atienden la caja (sin credenciales)	RF-CAJ-09
historialPrecio / historialCosto	Cambios de precio y de costo	RF-PRE-04
configuracion	Parámetros del negocio en pares clave-valor	RF-DAT-06

Definición de las tablas centrales

```

CREATE TABLE producto (
  id          INTEGER PRIMARY KEY,
  sku         TEXT    NOT NULL UNIQUE,
  nombre      TEXT    NOT NULL,
  categoria_id INTEGER REFERENCES categoria(id),
  unidad_base TEXT    NOT NULL CHECK (unidad_base IN
    ('unidad', 'kg', 'lb', 'g', 'L', 'ml')),
  es_granel   INTEGER NOT NULL DEFAULT 0 CHECK (es_granel IN (0,1)),
  -- Valor acumulado invertido, en millonésimas. Fuente de verdad del costo
  -- junto con la suma de existencias (RF-COS-02).
  valor_total INTEGER NOT NULL DEFAULT 0 CHECK (valor_total >= 0),
  stock_minimo INTEGER NOT NULL DEFAULT 0,
  activo      INTEGER NOT NULL DEFAULT 1 CHECK (activo IN (0,1)),
  creado_en   TEXT    NOT NULL
);

CREATE TABLE presentacion (
  id          INTEGER PRIMARY KEY,
  producto_id INTEGER NOT NULL REFERENCES producto(id),
  nombre      TEXT    NOT NULL,
  -- Milésimas: un six-pack es 6000. Debe ser > 0 (RF-PRS-14).
  factor      INTEGER NOT NULL CHECK (factor > 0),
  precio      INTEGER NOT NULL CHECK (precio >= 0),
  es_predeterminada INTEGER NOT NULL DEFAULT 0,
  codigo_barras TEXT,
  activa      INTEGER NOT NULL DEFAULT 1,
  UNIQUE (producto_id, nombre)
);

CREATE TABLE existencia (
  producto_id INTEGER NOT NULL REFERENCES producto(id),

```

```

ubicacion      TEXT      NOT NULL CHECK (ubicacion IN ('BODEGA','VITRINA')),
-- Milésimas de la unidad base. Nunca negativa (RF-INV-06).
cantidad       INTEGER NOT NULL DEFAULT 0 CHECK (cantidad >= 0),
PRIMARY KEY (producto_id, ubicacion)
);

CREATE TABLE movimiento_inventario (
  id            INTEGER PRIMARY KEY,
  ocurrido_en   TEXT      NOT NULL,
  producto_id   INTEGER NOT NULL REFERENCES producto(id),
  tipo          TEXT      NOT NULL CHECK (tipo IN
    ('ENTRADA','VENTA','TRASPASO','MERMA',
     'AJUSTE','DEVOLUCION')),
  ubicacion_origen TEXT,
  ubicacion_destino TEXT,
  cantidad      INTEGER NOT NULL,
  costo_unitario INTEGER NOT NULL, -- vigente en el momento del movimiento
  valor_movimiento INTEGER NOT NULL,
  motivo        TEXT,             -- obligatorio en AJUSTE y MERMA
  referencia_tipo TEXT,           -- 'VENTA' | 'COMPRA' | 'DEVOLUCION'
  referencia_id  INTEGER
);

CREATE TABLE venta_linea (
  id            INTEGER PRIMARY KEY,
  venta_id      INTEGER NOT NULL REFERENCES venta(id),
  producto_id   INTEGER NOT NULL REFERENCES producto(id),
  presentacion_id INTEGER NOT NULL REFERENCES presentacion(id),
  cantidad      INTEGER NOT NULL CHECK (cantidad > 0),
-- Los tres valores siguientes se CONGELAN al confirmar la venta
-- (RF-VTA-13, RF-PRS-13). Nunca se recalculan.
  factor_aplicado INTEGER NOT NULL,
  precio_aplicado INTEGER NOT NULL,
  costo_unitario  INTEGER NOT NULL,
  descuento       INTEGER NOT NULL DEFAULT 0,
  importe         INTEGER NOT NULL
);

CREATE TABLE sesion_caja (
  id            INTEGER PRIMARY KEY,
  fecha_negocio TEXT      NOT NULL, -- día local, lo define el dueño
  abierta_en    TEXT      NOT NULL,
  cerrada_en    TEXT,
  operador_id   INTEGER REFERENCES operador(id),
  fondo_inicial INTEGER NOT NULL,
-- Arqueo por moneda: nunca se consolidan (R-12)
  contado_cup   INTEGER,
  contado_usd   INTEGER,
  modo_cierre   TEXT      CHECK (modo_cierre IN ('SEPARADO','CONSOLIDADO')),
  tasa_cierre   INTEGER, -- tasa usada al consolidar
-- Comisión liquidada: inmutable una vez cerrada la sesión (RF-CMS-06)
  comision_porcentaje INTEGER,
  comision_base   INTEGER,
  comision_importe INTEGER,
  CHECK (cerrada_en IS NULL OR contado_cup IS NOT NULL)
);

CREATE TABLE venta (
  id            INTEGER PRIMARY KEY,
  folio         INTEGER NOT NULL UNIQUE,
  sesion_caja_id INTEGER NOT NULL REFERENCES sesion_caja(id),
  ocurrido_en   TEXT      NOT NULL,
  metodo_pago   TEXT      NOT NULL CHECK (metodo_pago IN

```

```

        ('EFFECTIVO_CUP','TRANSFERENCIA','EFFECTIVO_USD')),
-- Importe de la venta, SIEMPRE en pesos: no hay precios en divisa (R-4)
total          INTEGER NOT NULL,
-- Solo en cobros en dólares. La tasa se CONGELA aquí (RF-DIV-03):
-- si el dueño la actualiza mañana, esta venta no cambia.
tasa_aplicada   INTEGER,
entregado_divisa INTEGER,
vuelto          INTEGER NOT NULL DEFAULT 0,  -- siempre en pesos (R-11)
anulada         INTEGER NOT NULL DEFAULT 0,
CHECK ((metodo_pago = 'EFFECTIVO_USD') = (tasa_aplicada IS NOT NULL))
);

```

La última restricción no es cosmética: obliga a que todo cobro en dólares lleve su tasa y a que ningún cobro en pesos la lleve. Sin ella, una venta en divisa sin tasa sería irrecuperable, porque no habría forma de saber a cuánto se cobró.

Índices que sostienen RNF-1 y los informes:

```

CREATE INDEX idx_producto_nombre      ON producto(nombre);
CREATE INDEX idx_producto_categoria  ON producto(categoria_id);
CREATE INDEX idx_movimiento_producto ON movimiento_inventario(producto_id, ocurrido_en);
CREATE INDEX idx_movimiento_fecha    ON movimiento_inventario(ocurrido_en);
CREATE INDEX idx_venta_sesion        ON venta(sesion_caja_id);
CREATE INDEX idx_venta_fecha         ON venta(ocurrido_en);
CREATE INDEX idx_venta_linea_producto ON venta_linea(producto_id);

```

Búsqueda de productos (RNF-1). Un índice sobre `nombre` no resuelve la búsqueda por coincidencia parcial ni ignora acentos, que es lo que pide RF-CAT-09. Se añade una tabla de búsqueda de texto completo con FTS5 —incluido en SQLite— alimentada por disparadores, con el nombre normalizado sin acentos y en minúsculas. Es la diferencia entre recorrer 5 000 filas y no recorrerlas.

3.9 DT-9 — Transaccionalidad

Decisión. Cada caso de uso que modifica el estado se ejecuta dentro de una única transacción, abierta y cerrada en la capa de aplicación.

Confirmar una venta implica: insertar la venta, insertar sus líneas, descontar existencias, generar los movimientos de kárdex y actualizar el valor del inventario. **O pasa todo, o no pasa nada.** Es exactamente lo que exige RNF-5.

Las invariantes críticas se defienden en dos niveles, a propósito:

Invariante	En el dominio	En la base de datos
Existencia nunca negativa	Regla de negocio con mensaje entendible	CHECK (cantidad >= 0)
Factor de conversión positivo	Validación al crear la presentación	CHECK (factor > 0)
SKU único	Verificación previa	UNIQUE

La validación del dominio existe para dar un buen mensaje al usuario; la restricción de la base de datos existe porque un error de programación no debe poder corromper los datos. La redundancia es deliberada.

3.10 DT-10 — Migraciones de esquema

Decisión. Migraciones incrementales versionadas con `PRAGMA user_version`, aplicadas automáticamente al arrancar y siempre dentro de una transacción.

Cumple RF-DAT-08: el usuario actualiza la aplicación y sus datos se migran solos, sin pérdida. Si una migración falla, la transacción se revierte y la aplicación se niega a arrancar con un esquema a medias, en lugar de operar sobre datos corruptos.

Regla: las migraciones ya publicadas no se editan jamás. Un error se corrige con una migración nueva.

3.11 DT-11 — Respaldo y restauración

Decisión. El respaldo usa la API de copia en línea de SQLite, no una copia del archivo a nivel de sistema.

Por qué. Copiar el archivo mientras la aplicación lo tiene abierto —sobre todo en modo WAL— puede producir un respaldo inconsistente que parece válido hasta el día que se necesita. La copia en línea de SQLite garantiza una instantánea coherente.

Antes de restaurar (RF-DAT-07) se verifica que el archivo sea una base de datos válida, que su `user_version` sea compatible y que `PRAGMA integrity_check` pase. Un respaldo corrupto debe rechazarse **antes** de reemplazar los datos buenos, no después.

3.12 DT-12 — Compilación y distribución

Decisión. Integración continua con GitHub Actions y una matriz de dos sistemas operativos.

Sistema	Runner	Artefactos
Windows (principal)	windows-latest	.exe y .msi
macOS (secundaria)	macos-latest	.dmg universal (Intel y Apple Silicon)

Por qué no compilar Windows desde macOS. La compilación cruzada a Windows con Tauri es frágil: exige una cadena de herramientas adicional y se complica con WebView2 y la firma. Un runner de Windows real compila en su plataforma nativa y es reproducible. El equipo de desarrollo trabaja en macOS (Apple Silicon); sin este pipeline, no habría forma de producir el .exe que pide el negocio.

Motores web distintos. Windows usa WebView2 (Chromium) y macOS usa WebKit. No son equivalentes en representación visual, y por eso RNF-9 exige verificar en ambos antes de entregar.

Firma de código: no se firma. El uso previsto es personal, no hay distribución a terceros (ver DA-1, cerrada). Firmar costaría cerca de 99 USD al año para la notaría de Apple más un certificado aparte para Windows, y lo único que compra es eliminar advertencias que se saltan a mano una vez por versión.

Consecuencias asumidas al no firmar, y cómo se resuelven:

Situación	Qué ocurre	Cómo se resuelve
Compilar y ejecutar en la propia Mac	Nada. Gatekeeper no interviene en binarios compilados localmente.	No requiere acción.
Descargar el .dmg desde integración continua	Queda marcado con el atributo de cuarentena y Gatekeeper lo bloquea.	<i>Configuración del Sistema</i> → <i>Privacidad y seguridad</i> → <i>Abrir de todas formas</i> , o <code>xattr -d com.apple.quarantine <ruta></code> . En las versiones recientes de macOS el antiguo atajo de abrir con el botón secundario ya no basta.
Ejecutar el .exe descargado en Windows	SmartScreen muestra «Windows protegió su PC».	<i>Más información</i> → <i>Ejecutar de todas formas</i> . Reaparece con cada versión nueva, porque la evaluación es por reputación del binario.

WebView2 en Windows. La aplicación depende de WebView2 para representar la interfaz. Viene preinstalado en Windows 11 y en Windows 10 actualizado, pero no puede darse por supuesto. El instalador debe configurarse para incorporar el instalador de WebView2 y resolverlo por su cuenta, tal como exige RNF-10.

3.13 DT-13 — Nomenclatura: el dominio habla español

Decisión. El vocabulario del negocio se escribe en español. La estructura técnica y las convenciones propias de cada lenguaje se mantienen en inglés.

Por qué. Los términos del negocio no tienen traducción limpia: *bodega* y *vitrina* son ambas `warehouse/display` según a quién se le pregunte, *merma* no es exactamente `waste` ni `shrinkage`, y *kárdex* o *arqueo* directamente no tienen equivalente de uso corriente. Traducirlos obliga a mantener un diccionario mental entre lo que dice el dueño del mercadito y lo que dice el código, y ese diccionario es donde se cuelan los malentendidos. El código debe poder leerse junto al documento de requerimientos sin traducir nada.

Qué va en español

Elemento	Ejemplo
Entidades y objetos de valor	Producto, Presentacion, Existencia, SesionCaja
Campos y variables del dominio	costo_unitario, valor_total, factor_conversion
Casos de uso	RegistrarVenta, ReabastecerVitrina, CerrarCaja
Puertos y sus implementaciones	RepositorioProducto, RepositorioProductoSqlite
Tablas y columnas de la base de datos	movimiento_inventario, ubicacion
Comandos expuestos a la interfaz	registrar_venta, cerrar_caja
Carpetas de funcionalidad en Angular	features/venta/, features/vitrina/
Comentarios y documentación	Todo

Qué va en inglés

Elemento	Motivo
Nombres de los crates: domain, application, infrastructure	Son términos de arquitectura, no del negocio; se reconocen universalmente
Sufijos técnicos de framework: ProductoService, VentaPageComponent	Convención de Angular
Todo lo propio del lenguaje: Result, Error, impl, new, from	Convención de Rust
Mensajes de commit y descripciones de PR	Convención del equipo

Regla obligatoria: identificadores sin tildes ni eñes. Rust y TypeScript admiten Unicode en identificadores, pero usarlo es fuente de errores difíciles de ver: `sesión` y `sesion` son dos identificadores distintos y en pantalla casi idénticos. Se escribe `sesion_caja`, `numero`, `anio`. Las tildes y eñes sí se usan —y se deben usar— en comentarios, mensajes de error y textos de interfaz.

4 Estructura del repositorio

kilo12-app/	
├ docs/	Requerimientos y diseño (fuente .md + PDF)
├ scripts/	build-pdf.py y utilidades de desarrollo
├ crates/	
│ └ domain/	Reglas de negocio puras
│ └ application/	Casos de uso y puertos
│ └ infrastructure/	Adaptador SQLite, migraciones, respaldo
├ src-tauri/	Comandos, ensamblado, configuración de Tauri
├ ui/	Aplicación Angular
└ .github/workflows/	Integración continua y publicación

5 Modelo de dominio

Entidades y objetos de valor principales. Los tipos son ilustrativos del diseño, no código definitivo.

Concepto	Tipo	Invariante que protege
Dinero	Objeto de valor sobre entero escalado	Nunca se construye desde un flotante
Cantidad	Objeto de valor con su unidad	No se suman cantidades de unidades distintas
Producto	Entidad	Tiene al menos una presentación (RF-PRS-03)
Presentacion	Entidad	Factor positivo; entero si la unidad base es unidad (RF-PRS-14)
Existencia	Objeto de valor por ubicación	Nunca negativa (RF-INV-06)
Valoracion	Servicio de dominio	Recalcula el costo solo en entradas (RF-COS-04)
Venta	Agregado	Al confirmarse congela precio, factor y costo
SesionCaja	Agregado	Una vez cerrada es inmutable (RF-CAJ-08)

El agregado venta es la pieza más delicada del sistema. Confirmar una venta coordina: validar existencia en vitrina convirtiendo presentaciones a unidad base, descontar, congelar tres valores por línea, generar kárdex, actualizar el valor del inventario y asociar la venta a la sesión de caja abierta. Todo dentro de una transacción, y todo sujeto a los casos de prueba que ya están escritos en los requerimientos.

6 Estrategia de pruebas

Nivel	Qué cubre	Cómo
Dominio	Valoración, presentaciones, comisión, invariantes	Pruebas unitarias puras, sin base de datos
Aplicación	Casos de uso completos	Puertos sustituidos por implementaciones en memoria
Infraestructura	Consultas, migraciones, restricciones	SQLite en memoria
Contrato	Serialización de comandos y errores	Pruebas de los objetos de transferencia
Interfaz	Componentes de presentación	Pruebas de componente de Angular
Extremo a extremo	Venta completa con teclado	Manual en ambas plataformas antes de entregar (RNF-9)

Los requerimientos ya traen las pruebas escritas. Tres casos numéricos que deben implementarse antes que el código que validan:

1. **Arroz** (§6.6): 100 kg @ \$10 → venta 40 → 100 kg @ \$14 produce costo \$12.50 sobre 160 kg; la venta posterior de 50 kg deja ganancia bruta de \$275. 2. **Refresco** (§6.2): unidad a \$80 y six-pack a \$300 sobre costo base \$41.67; vender un six-pack descuenta 6 unidades. 3. **Comisión** (§6.11): venta de \$100 000 al 2 % da \$2 000, y la ganancia neta de comisión es \$28 000.

7 Riesgos técnicos

ID	Riesgo	Impacto	Mitigación
RT-1	Diferencias visuales entre WebView2 y WebKit	Interfaz correcta en una plataforma y rota en la otra	RNF-9: verificar en ambas antes de cada entrega; evitar CSS experimental
RT-2	Pérdida de precisión al cruzar la frontera a Angular	Descuadres silenciosos en dinero	DT-7: montos como texto; prohibida la aritmética monetaria en el frontend
RT-3	El equipo del mercadito falla y no hay respaldo reciente	Pérdida total del historial	RF-DAT-04 y RF-DAT-05: respaldos automáticos y recordatorio
RT-4	Una migración defectuosa corrompe datos productivos	Datos irrecuperables	DT-10: migración transaccional; respaldo automático previo a migrar
RT-5	Advertencias de SmartScreen o Gatekeeper al instalar	El propietario desconfía de su propia aplicación	DT-12: decidir firma antes de distribuir; documentar el procedimiento de instalación
RT-6	La ceremonia de cuatro crates frena el desarrollo	Costo sin retorno en un proyecto pequeño	Revisar tras el primer caso de uso completo; fusionar capas si no aporta

8 Decisiones

8.1 Cerradas

ID	Decisión	Resolución	Justificación
DA-1	Firma de código del ejecutable	No se firma	El uso es personal y no hay distribución a terceros. Las advertencias de SmartScreen y Gatekeeper se salvan manualmente una vez por versión y no impiden ejecutar la aplicación. El gasto anual no compra ninguna funcionalidad. Se revisará si algún día la aplicación se entrega a otros mercaditos. Detalle en DT-12.
DA-2	Idioma de los identificadores del código	Dominio en español; estructura y convenciones de lenguaje en inglés	El vocabulario del negocio —bodega, vitrina, merma, kárdex, arqueo— carece de traducción limpia, y traducirlo obliga a mantener un diccionario mental entre lo que dice el dueño y lo que dice el código. Detalle y alcance exacto en DT-13.

8.2 Abiertas

ID	Decisión	Cuándo resolverla
DA-3	Si el respaldo automático se dispara al cerrar la aplicación o por programación	Al implementar RF-DAT-04

9 Trazabilidad: requerimiento → decisión

Requerimiento	Decisión que lo sostiene
R-1, R-2 (offline, sin servidor)	DT-2, DT-3
R-6, RF-DAT-02/03/07 (respaldo)	DT-11
RNF-1 (búsqueda < 100 ms)	DT-8 (índices y búsqueda de texto completo)
RNF-4 (periféricos futuros)	DT-5 (puertos y adaptadores)
RNF-5 (atomicidad)	DT-2, DT-9
RNF-6 (decimal exacto)	DT-1, DT-4, DT-7
RNF-8, RNF-9, RNF-10 (plataformas)	DT-12
RF-COS-02/04/09 (valoración)	DT-4, DT-5
RF-VTA-13, RF-PRS-13 (congelado)	DT-8 (columnas congeladas en venta_linea)
RF-CAJ-08, RF-CMS-06 (inmutabilidad)	DT-8 (sesion_caja), DT-9
RF-DAT-08 (migraciones)	DT-10